

Unit 6 - Treap

CS 201 - Data Structures II

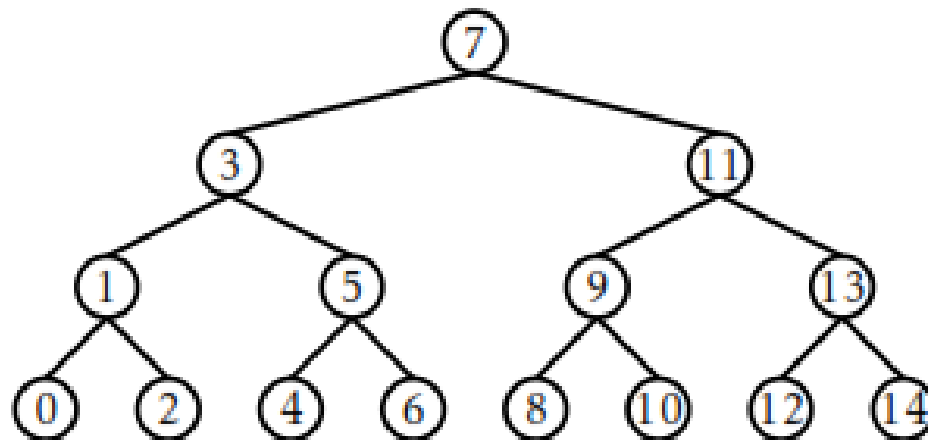
Spring 2024

Habib University

Syeda Saleha Raza

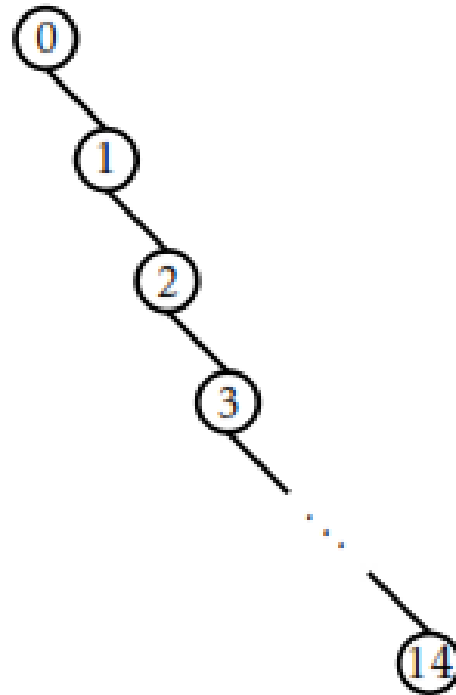
Is this a BST?

$\langle 7, 3, 11, 1, 5, 9, 13, 0, 2, 4, 6, 8, 10, 12, 14 \rangle$.



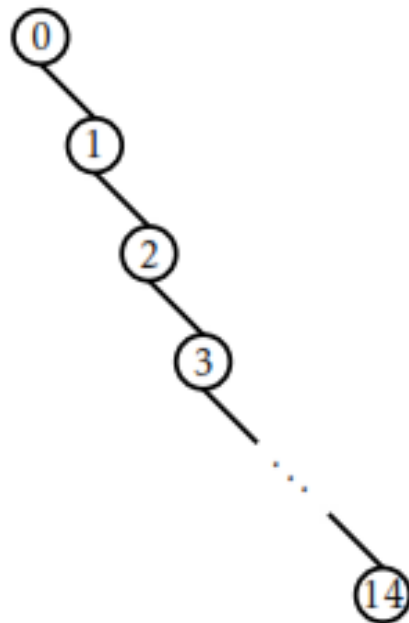
Is this a BST?

$\langle 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 \rangle$.

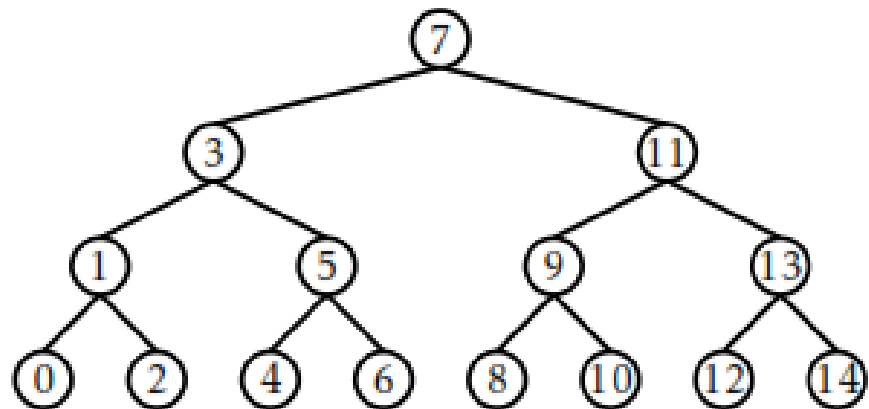


Binary Search Tree on same data

$\langle 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 \rangle$.



$\langle 7, 3, 11, 1, 5, 9, 13, 0, 2, 4, 6, 8, 10, 12, 14 \rangle$.



Randomness in BST

Theorem 7.1. *A random binary search tree can be constructed in $O(n \log n)$ time. In a random binary search tree, the $\text{find}(x)$ operation takes $O(\log n)$ expected time.*

Treap

(A blend of Tree + Heap)

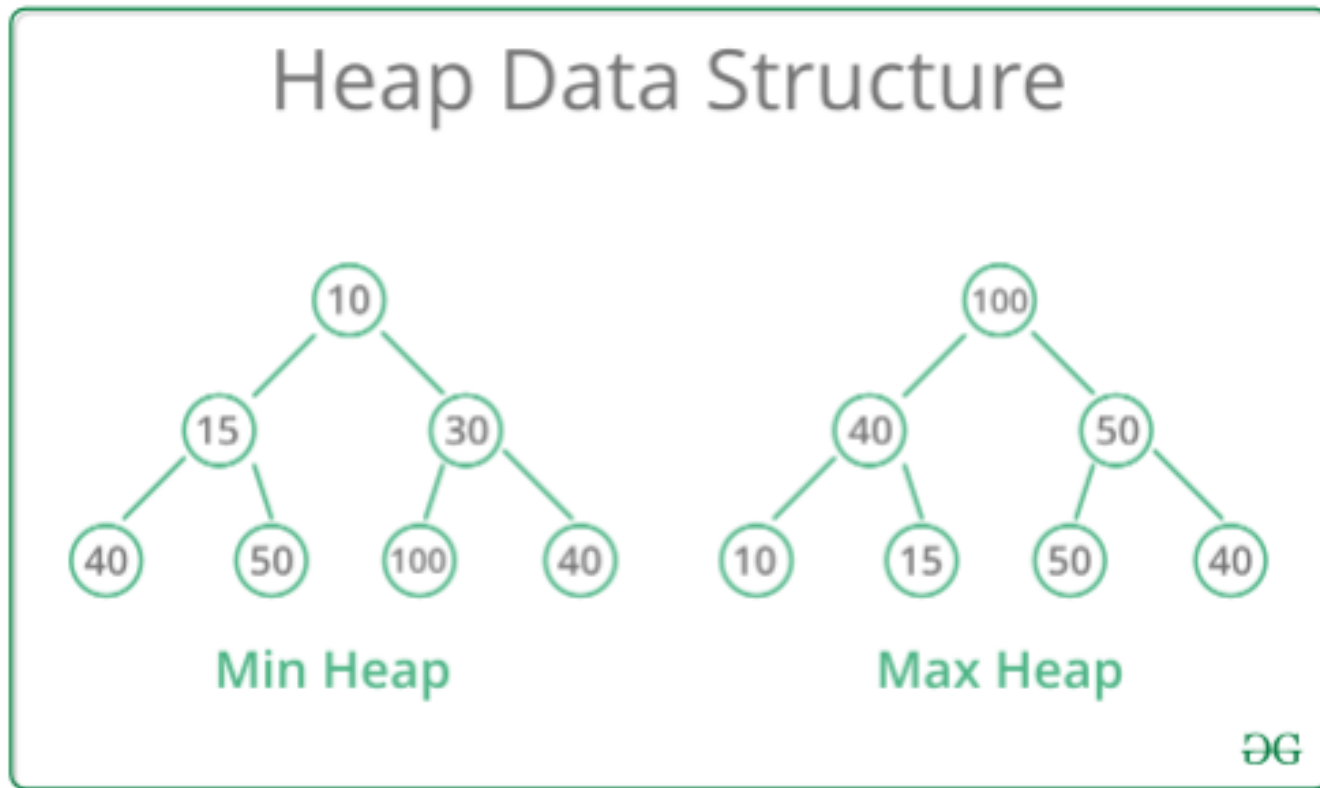
Treap - Randomized Binary Search Tree

Every node of Treap maintains two values:

1. **Key** Follows standard BST ordering (left is smaller and right is greater)
2. **Priority** Randomly assigned value that follows Max-heap/Min-heap property.

[Treap \(A Randomized Binary Search Tree\) - GeeksforGeeks](#)

Heap Property



Heap Property

- Each node has a priority smaller than that of its two children. An example is shown in Figure below.

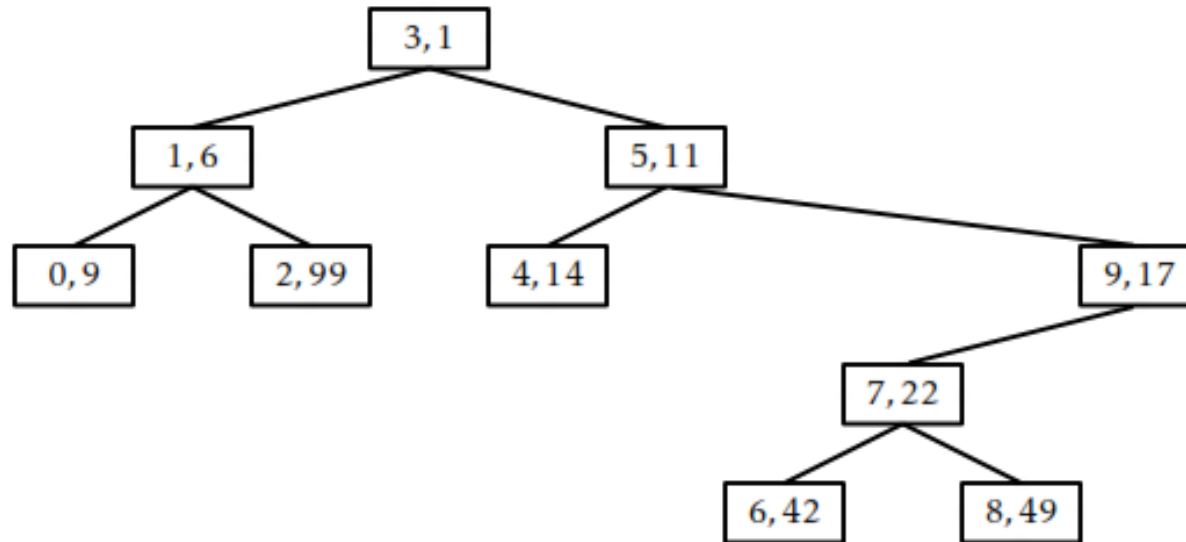


Figure 7.5: An example of a Treap containing the integers $0, \dots, 9$. Each node, u , is illustrated as a box containing $u.x, u.p$.

Treap Properties

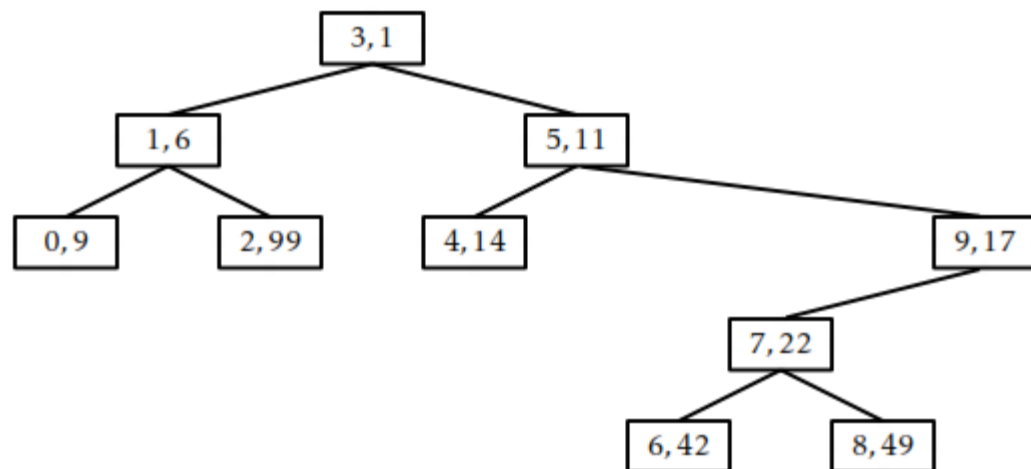
- The **heap property** tells us that the node with minimum priority has to be the root, r , of the Treap.
- The **binary search tree property** tells us that all nodes with keys smaller than the current key are stored in the left subtree rooted and all nodes with keys larger than the current key are stored in the right subtree.
- The important point about the priority values in a Treap is that they are **unique** and **assigned at random**.
- The heap and binary search tree conditions together to ensure that **the shape of the Treap is completely determined**.

Treap

- We can think of a Treap as a:
 - Tree that obeys heap and binary search property.
 - Binary Search Tree whose nodes were added in increasing order of priority.

$\langle (3, 1), (1, 6), (0, 9), (5, 11), (4, 14), (9, 17), (7, 22), (6, 42), (8, 49), (2, 99) \rangle$

$\langle 3, 1, 0, 5, 9, 4, 7, 6, 8, 2 \rangle$



Tree Rotation

- Tree rotation is a transformation technique which can be used to change the structure of the binary search tree *without changing the order of the elements*.
- Rotation supports in balancing the by reducing the depth of a node by one and increases the depth of its parent by one.

Rotation in Tree

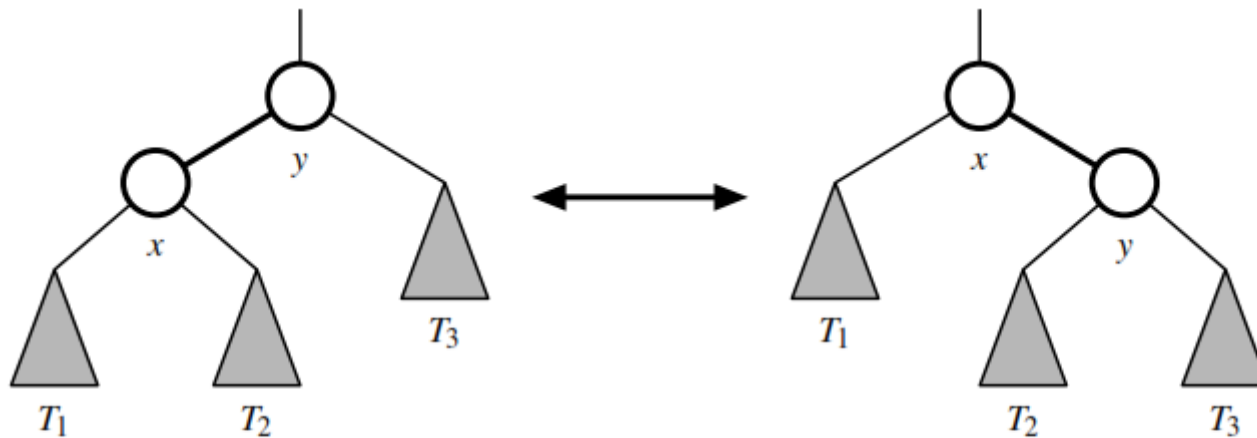
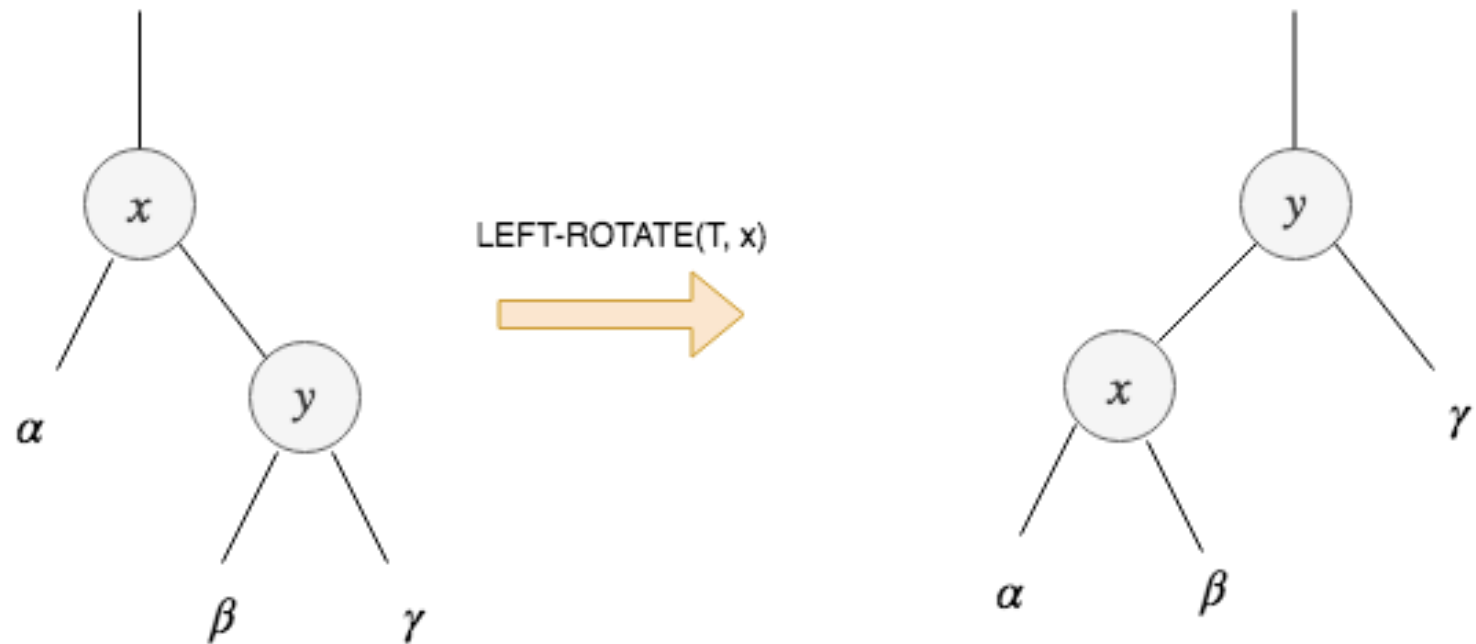


Figure 11.8: A rotation operation in a binary search tree. A rotation can be performed to transform the left formation into the right, or the right formation into the left. Note that all keys in subtree T_1 have keys less than that of position x , all keys in subtree T_2 have keys that are between those of positions x and y , and all keys in subtree T_3 have keys that are greater than that of position y .

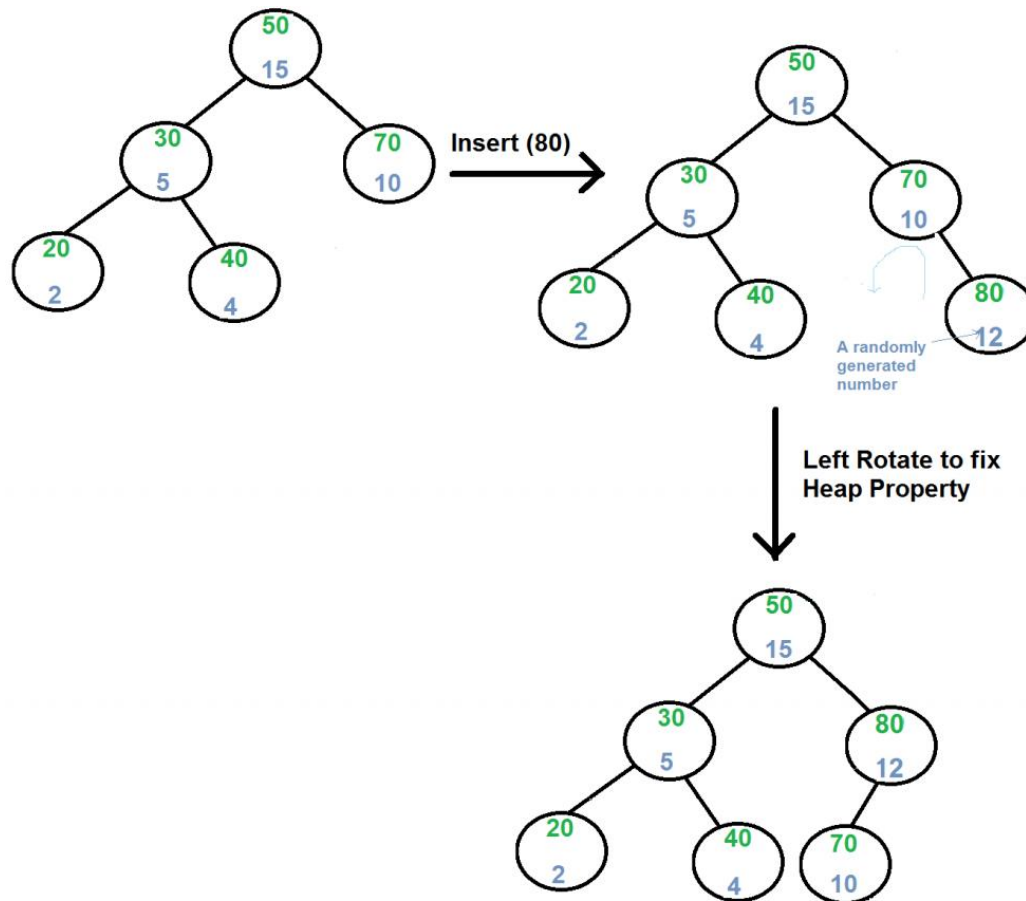
Left Rotation



Right Rotation



Insertion in Treap



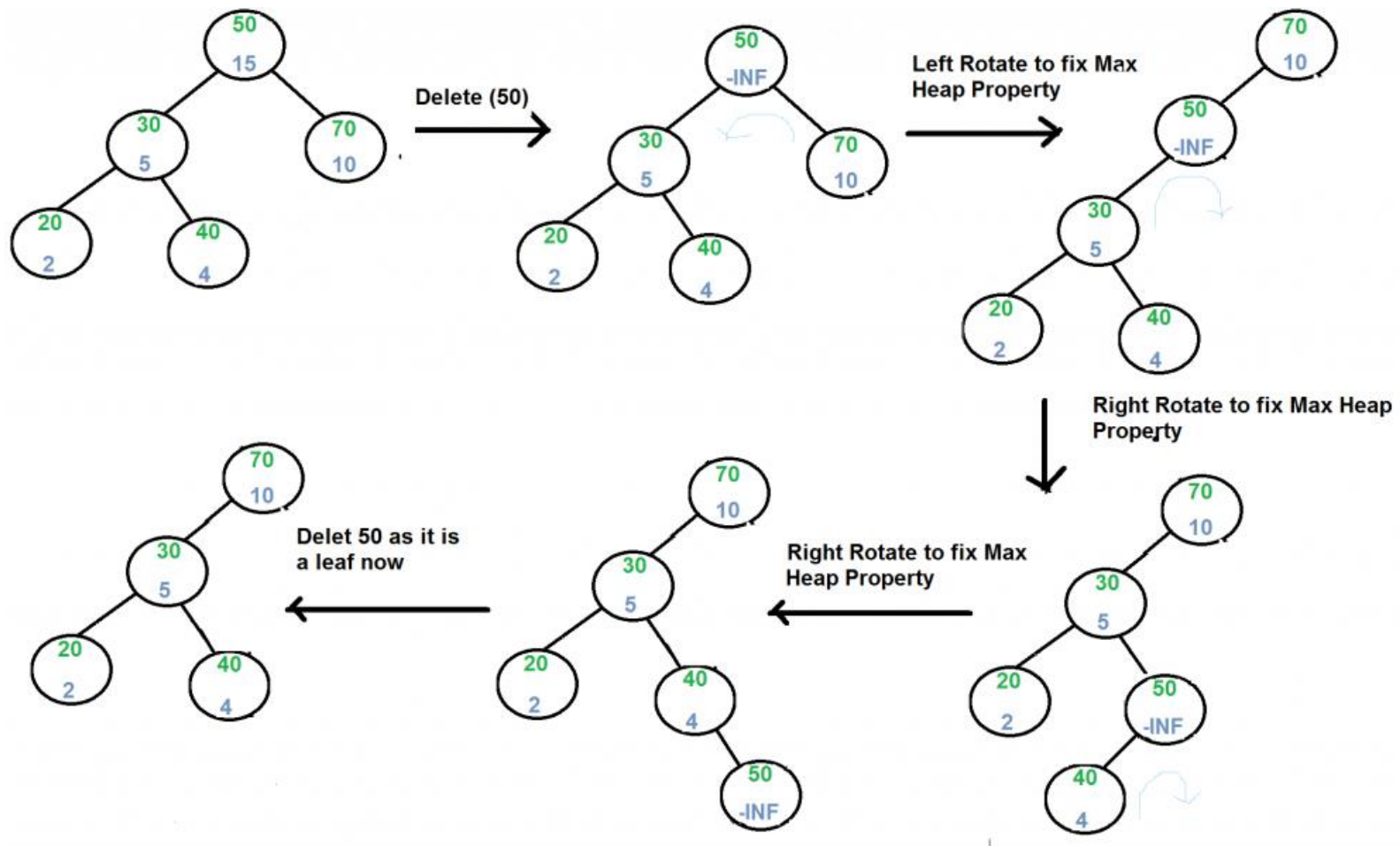
Exercise – Building a treap

- Let's take the extreme case in which all the numbers are in order:
 - Keys: 0,1,2,3,4, 5, 6,7,8
 - Priority: 5,9,2,8,7, 4, 9,1,3

Deletion in Treap

- If node to be deleted is a leaf, delete it.
- Else replace node's priority with minus infinite ($-\text{INF}$), and do appropriate rotations to bring the node down to a leaf.
- If the node(to be deleted) has two child nodes, rotate it with the child that has larger priority to preserve heap ordering.

Deletion in Treap



Complexity

Theorem 7.1. *A random binary search tree can be constructed in $O(n \log n)$ time. In a random binary search tree, the `find(x)` operation takes $O(\log n)$ expected time.*

Complexity

Theorem 7.2. *A Treap implements the SSet interface. A Treap supports the operations `add(x)`, `remove(x)`, and `find(x)` in $O(\log n)$ expected time per operation.*

Treap vs Skiplist

- Both can be used for SSet interface
- Find(), add(), remove()

Resources

- Open Data Structures (pseudocode edition), by Pat Morin. Available online at <http://opendatastructures.org>
- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing
- <https://www.geeksforgeeks.org/treap-a-randomized-binary-search-tree/>